Univerzitet u Sarajevu
Elektrotehnički fakultet
Odsjek za računarstvo i informatiku

Layout Algebra

ZAVRŠNI RAD

– PRVI CIKLUS STUDIJA –

Mentor:
Prof. dr. Vensada Okanović

Student:
Nedim Omanović

Sarajevo, mart 2026.

Elektrotehnički fakultet, Univerzitet u Sarajevu
Odsjek za računarstvo i informatiku
Prof. dr. Vensada Okanović, dipl. ing. el.
Sarajevo, mart 2026.

Postavka zadatka završnog rada I ciklusa:

Layout Algebra – Alat za upravljanje rasporedom elemenata web stranice zasnovan na ograničenjima

Ideja je da se kreira alat koji omogućava formiranje layout-a web stranice na osnovu skupa ograničenja, varijabli, operatora i jednačina relevantnih za izgled web stranice.

- **Ograničenja** definiraju pravila i odnose koji određuju kako elementi trebaju biti raspoređeni u layout-u. Na primjer, ograničenje može odrediti da dugme treba biti horizontalno centrirano u nekom kontejnerskom elementu ili da dva elementa trebaju biti poravnata na svojim vrhovima.
- **Varijable** predstavljaju dimenzije, položaje i veličine elemenata u izgledu. Na primjer, širina i visina prozora ili pozicija dugmeta se mogu predstaviti kao varijable.
- **Operatori** se koriste za izražavanje odnosa i operacija između varijabli i ograničenja. Ovi operatori mogu uključivati: sabiranje, oduzimanje, množenje, dijeljenje, poređenje i slično.
- **Jednačine** definiraju odnose između ograničenja i varijabli. One izražavaju kako se određeno ograničenje ili varijabla izračunava na temelju drugih varijabli, ograničenja ili konstanti. Jednačine pomažu u održavanju dosljednosti u izgledu.

Polazna literatura:

[1] Badros, G.J., Borning, A., Stuckey, P.J., "The Cassowary Linear Arithmetic Constraint Solving Algorithm", ACM Transactions on Computer-Human Interaction, 2001.

[2] Marriott, K., Stuckey, P.J., "Programming with Constraints: An Introduction", MIT Press, 1998.

Prof. dr. Vensada Okanović, dipl. ing. el.

Ovdje ubaciti i popuniti formular Izjave o autentičnosti radova propisan od strane Univerziteta u Sarajevu.

Sažetak

Ovaj rad predstavlja razvoj interaktivnog alata za vizualno upravljanje rasporedom (layoutom) elemenata web stranice zasnovanog na konceptu programiranja ograničenjima (constraint programming). Implementirani sistem nosi naziv **Layout Algebra** i omogućava korisnicima definisanje rasporeda elemenata web stranice putem namjenskog domenski-specifičnog jezika (DSL – Domain-Specific Language), koji uključuje pojmove poput varijabli, ograničenja (constraints), operatora, hijerarhija elemenata i pravila izbjegavanja preklapanja.

Osnovna ideja sistema jeste da korisnik ne mora ručno unositi apsolutne koordinate i dimenzije elemenata, već umjesto toga opisuje **logičke odnose** između elemenata: npr. da jedan element treba biti horizontalno centriran u odnosu na kontejner, da mu širina treba biti polovina širine drugog elementa, ili da dva elementa ne smiju preklapati jedan drugog. Ova ograničenja se potom automatski rješavaju pomoću matematičkog solvera zasnovanog na Cassowary algoritmu, koji pronalazi pozicije i dimenzije elemenata koji zadovoljavaju sva zadana ograničenja.

Sistem je implementiran kao web aplikacija u programskom jeziku TypeScript, koristeći React biblioteku za korisnički interfejs, Vite kao alat za izgradnju projekta te Zustand biblioteku za upravljanje stanjem aplikacije. Za rješavanje sistema ograničenja koristi se biblioteka *kiwi.js*, koja implementira Cassowary algoritam linearnog programiranja ograničenja. Aplikacija nudi dvosmjernu interakciju između tekstualnog DSL editora i vizualnog kanvasa.

Pored vizualnog prikaza, sistem podržava grupisanje elemenata, upravljanje hijerarhijskim odnosima roditelj-dijete, undo/redo i napredno upravljanje historijom akcija te modul za export generisanog rasporeda u standardne CSS formate: apsolutno pozicioniranje, Flexbox, CSS Grid te kompletan HTML fajl.

Ključne riječi: DSL, constraint programming, Cassowary, layout management, web dizajn, React, TypeScript, Kiwi.js.

Abstract

This thesis presents the development of an interactive web page layout management tool based on constraint programming. The implemented system, named **Layout Algebra**, allows users to define element layouts using a custom Domain-Specific Language (DSL) that incorporates variables, constraints, operators, element hierarchies and overlap-avoidance rules.

The core idea is that the user does not need to manually enter absolute coordinates; instead, they describe **logical relationships** between elements. These constraints are automatically resolved using a mathematical solver based on the Cassowary algorithm.

The system is implemented as a TypeScript web application using React, Vite and Zustand. The *kiwi.js* library implements the Cassowary linear constraint-programming algorithm. The application provides bidirectional interaction between the text-based DSL editor and a visual canvas. A CSS export module generates layouts in Absolute, Flexbox, Grid and HTML formats.

Keywords: DSL, constraint programming, Cassowary, layout management, web design, React, TypeScript, Kiwi.js.

Sadržaj

Sažetak **3**

Abstract **4**

Popis slika **7**

Popis tabela **8**

Uvod **9**

1 Teorijske osnove **11**
1.1 Programiranje ograničenjima 11
1.1.1 Historijski pregled i primjene 11
1.1.2 Vrste sistema ograničenja 12
1.2 Cassowary algoritam 13
1.2.1 Hijerarhija prioriteta ograničenja 13
1.2.2 Simpleks metoda kao osnova 14
1.2.3 Edit varijable i mehanizam sugestija 14
1.2.4 Kiwi.js biblioteka 15
1.3 Domenski-specifični jezici (DSL) 16
1.3.1 Principi dizajna dobrog DSL-a 16
1.3.2 DSL u kontekstu upravljanja layoutom 16
1.3.3 Poređenje s alternativnim DSL pristupima za layout 17
1.4 React i moderna web arhitektura 18
1.4.1 Zustand – upravljanje stanjem 18
1.4.2 Vite – alat za izgradnju 18
1.4.3 TypeScript – sistem tipova 19

2 Realizacija sistema Layout Algebra **20**
2.1 Arhitektura sistema 20
2.2 DSL parser 21
2.2.1 Podržane komande 22
2.2.2 Parsiranje ograničenja 22
2.2.3 Rukovanje greškama u parseru 24
2.3 Solver – rješavač ograničenja 25
2.3.1 Strukturalna ograničenja 25

2.3.2 Generisanje kiwi.js ograničenja iz AST-a 25
2.3.3 Implementacija prevlačenja 26
2.3.4 Implementacija AVOID pravila 27
2.4 Upravljanje stanjem aplikacije 28
2.4.1 Reaktivni tok podataka 28
2.4.2 Undo/Redo mehanizam 28
2.4.3 Grupisanje elemenata 29
2.5 Korisnički interfejs 30
2.5.1 Vizualni kanvas 31
2.5.2 DSL editor i Constraint Builder 31
2.5.3 Panel elemenata i hijerarhija 32
2.6 Export modul 33
2.6.1 Apsolutno pozicioniranje 33
2.6.2 CSS Grid eksport 33
2.6.3 Flexbox eksport 34

3 Rezultati i analiza 35
3.1 Primjeri upotrebe 35
3.1.1 Primjer 1: Dashboard layout 35
3.1.2 Primjer 2: Forma s postotnim dimenzijama i ugniježdenim elementima 36
3.1.3 Primjer 3: Izbjegavanje preklapanja s hijerarhijom prioriteta . . 38
3.1.4 Primjer 4: Višerazinska hijerarhija – landing page 40
3.2 Analiza performansi 43
3.3 Poređenje s alternativnim pristupima 44
3.4 Ograničenja implementacije i diskusija 45

Zaključak 46

A Sintaksa Layout Algebra DSL 47

B Pregled strukture projekta 49

Literatura 50

Popis slika

1.1 Tok rješavanja ograničenja – od AST do izračunatih pozicija putem Cassowary/kiwi.js algoritma 15

2.1 Arhitektura sistema Layout Algebra – prikaz slojeva i toka podataka između komponenata 20
2.2 Tok parsiranja DSL koda – transformacija tekstualnog ulaza u AST strukturu 22
2.3 Korisnički interfejs aplikacije Layout Algebra sa DSL editorom, komponentama, vizualnim kanvasom i panelom osobina 31

3.1 Rezultat Primjera 1 – dashboard layout s topbarom, sidebarom, tri kartice u redu, grafikonom i tabelom 36
3.2 Rezultat Primjera 2 – forma s postotnim dimenzijama, ugniježdenim elementima i info panelom 38
3.3 Rezultat Primjera 3 – tri tooltipa s AVOID pravilom i hijerarhijom prioriteta; solver automatski rješava kolizije i granice kontejnera 40
3.4 Rezultat Primjera 4 – landing page s tri nivoa hijerarhije: navbar, hero sekcija i grid kartica s ugniježdenim ikonama 42

Popis tablica

2.1 Pregled komandi Layout Algebra DSL 22
2.2 Podržane karakteristike elemenata u ograničenjima 22
2.3 Nivoi prioriteta ograničenja 26
2.4 Podržani formati eksporta 33

3.1 Izmjerena brzina rješavanja u zavisnosti od broja elemenata 43
3.2 Poređenje pristupa za definisanje layouta 44

B.1 Struktura projekta Layout Algebra 49
B.2 Ključne zavisnosti projekta 49

Uvod

Definisanje vizualnog rasporeda elemenata (layouta) web stranice jedan je od centralnih problema u razvoju korisničkih interfejsa. Tradicionalni pristupi – kao što su apsolutno pozicioniranje u CSS-u, Flexbox i CSS Grid – korisnicima nameću razmišljanje u terminima koordinatnog sistema ili specifičnih jezičkih apstrakcija. Svaki od ovih pristupa ima prednosti i nedostatke; apsolutno pozicioniranje je intuitivno, ali lomljivo i teško održivo, dok Flexbox i Grid nude snažne alate za responzivni dizajn, ali zahtijevaju dobro poznavanje CSS-a.

U praksi, web programeri provode značajan dio vremena upravo na rješavanju problema layouta – poravnanju elemenata, centriranju, održavanju proporcija pri različitim veličinama ekrana i sličnom. Istraživanja pokazuju da layout problemi spadaju među najčešće izazove s kojima se suočavaju front-end developeri početnici, ali i iskusni stručnjaci koji rade sa složenim UI sistemima [1].

Drugačiji pristup nudi **programiranje ograničenjima** (constraint programming), gdje korisnik umjesto eksplicitnih koordinata opisuje *odnose* između elemenata. Na primjer: „element A je horizontalno centriran unutar kontejnera B”, ili „element C ima istu širinu kao element D, uvećanu za 50 piksela”. Sistem automatski rješava takav sistem ograničenja i pronalazi konkretne vrijednosti pozicija i dimenzija. Ovaj pristup koriste GUI sistemi poput Apple AutoLayout-a (iOS/macOS) i historijskog TeX sistema. Međutim, analogni alati za web dizajn ostaju rijetki i uglavnom istraživački.

Cilj ovog rada jeste dizajn i implementacija interaktivnog alata pod nazivom **Layout Algebra**, koji primjenjuje programiranje ograničenjima za raspoređivanje elemenata web stranice. Ključni doprinos je uvođenje namjenskog domensko-specifičnog jezika (DSL) koji je razumljiv, proširiv i dovoljan za opisivanje raznolikih scenarija rasporeda. Uz DSL, alat nudi potpuno interaktivni vizualni editor s dvosmjernom sinhronizacijom: korisnik može i pisati DSL kod i direktno manipulisati elementima na kanvasu mišem.

U toku izrade rada korištene su sljedeće metode i tehnologije:

- Dizajn i implementacija custom DSL-a s odgovarajućim parserom
- Integracija Cassowary solvera za rješavanje linearnih ograničenja (*kiwi.js*)
- Implementacija React web aplikacije s modernom arhitekturom (Zustand, Vite)
- Implementacija vizualnog editora s podrškom za prevlačenje, promjenu veličine, grupisanje i undo/redo
- Implementacija export modula za standardne CSS formate

Dostignuti rezultati uključuju funkcionalan prototip alata koji rješava ograničenja

u realnom vremenu, podržava hijerarhijske odnose roditelj-dijete između elemenata, omogućava grupno prevlačenje elemenata i eksportuje rezultat u standardni CSS/HTML.

Rad je organiziran na sljedeći način: u poglavlju 1 izlažu se teorijske osnove – programiranje ograničenjima, Cassowary algoritam, principi DSL dizajna i korišteni tehnološki stack. Poglavlje 2 detaljno opisuje arhitekturu i implementaciju svih komponenti sistema. Poglavlje 3 prikazuje konkretne primjere upotrebe i analizira performanse i ograničenja rješenja. Na kraju se donosi zaključak i daju smjernice za budući razvoj.

1 Teorijske osnove

1.1 Programiranje ograničenjima

Programiranje ograničenjima (Constraint Programming, CP) je paradigma rješavanja problema u kojoj se rješenje opisuje kroz skup ograničenja nad varijablama, a ne eksplicitnim algoritmom za pronalaženje rješenja [1]. Problem se formuliše kao trojka (V, D, C) gdje je V skup varijabli, D domena svake varijable, a C skup ograničenja koja moraju biti zadovoljena.

Definicija 1.1 (Sistem ograničenja)**.** *Sistem ograničenja je trojka (V, D, C) gdje je $V = \{x_1, x_2, \ldots, x_n\}$ skup varijabli, $D = \{D_1, D_2, \ldots, D_n\}$ skup domena (pri čemu D_i označava skup dopustivnih vrijednosti za varijablu x_i), a $C = \{c_1, c_2, \ldots, c_m\}$ skup ograničenja. Rješenje sistema je dodjela vrijednosti $x_i \in D_i$ svim varijablama takva da su sva ograničenja iz C zadovoljena.*

Ova paradigma se više od pola vijeka uspješno primjenjuje u različitim domenima – od planiranja i rasporedivanja u industriji, optimizacije transportnih mreža, bioinformatike, do domena koji je relevantan za ovaj rad: interaktivnih grafičkih korisničkih interfejsa. Ključna prednost ovog pristupa leži u jasnom razdvajanju *šta* se želi postići od *kako* se to postiže – korisnik opisuje željeno stanje sistema, a solver brine o pronalaženju puta do tog stanja.

Za primjenu u grafičkim korisničkim interfejsima posebno su relevantna **linearna ograničenja**, tj. ograničenja oblika:

$$a_1x_1 + a_2x_2 + \cdots + a_nx_n \ \leq / = / \geq \ b$$

gdje su x_i varijable (npr. koordinata lijevog ruba elementa), a_i koeficijenti, a b konstanta. Ovakvi sistemi mogu se efikasno rješavati algoritmima linearnog programiranja.

1.1.1 Historijski pregled i primjene

Rana primjena koncepta ograničenja u GUI sistemima datira iz rada Sketchpad sistema Ivana Sutherlanda iz 1963. godine [2]. Sketchpad je bio revolucionaran sistem koji je korisnicima dozvoljavao da crtaju geometrijske figure postavljanjem odnosa između njih – paralelnost, okomitost, jednakost dužina – a sistem je automatski održavao te odnose čak i kada bi korisnik pomjerao elemente. Ovaj rad je postavio temelje za čitav niz kasnijih istraživanja.

U decenijama koje su uslijedile pojavilo se više istraživačkih sistema koji su eksperimentisali s ograničenjima u GUI okruženjima. Posebno treba izdvojiti sistem Thinglab Alana Borninga, koji je još 1979. godine demonstrirao višesmjerne ograničenje na objektima u Smalltalk okruženju, te Garnet sistem sa Univerziteta Carnegie Mellon koji je u kasnim 1980-im godinama implementirao napredne layout ograničenja.

Danas najvažnija komercijalna primjena sistema ograničenja u GUI domenima jeste Apple AutoLayout, koji je uveden u iOS 6 (2012. godine) i koji se temelji direktno na Cassowary algoritmu. AutoLayout je u potpunosti zamijenio stariji način definiranja layouta putem *autoresizing masks* i danas je standardni način definiranja korisničkih interfejsa u iOS i macOS ekosistemu.

1.1.2 Vrste sistema ograničenja

Ovisno o prirodi varijabli i ograničenja, razlikujemo nekoliko klasa sistema ograničenja. Za kontekst ovog rada relevantne su dvije klase:

Sistemi linearnih aritmetičkih ograničenja nad realnim brojevima bave se ograničenjima oblika $a_1x_1 + \ldots + a_nx_n \ \{\leq, =, \geq\} \ b$ gdje su sve varijable iz skupa realnih ili racionalnih brojeva. Ovi sistemi imaju dobro razvijenu teoriju (simpleks metoda, metod unutrašnjih tačaka) i efikasne algoritme koji se koriste i u ovom radu.

Sistemi ograničenja nad konačnim domenima (CSP – Constraint Satisfaction Problems) bave se varijablama koje mogu uzeti vrijednosti iz konačnog skupa (npr. boje, dani u sedmici). Tipični algoritmi su backtracking s tehnikama propagacije ograničenja (arc consistency, forward checking). Iako su CSP sistemi značajni u širem kontekstu problema ograničenja, u ovom radu fokus je isključivo na linearnim aritmetičkim ograničenjima nad realnim brojevima.

1.2 Cassowary algoritam

Cassowary je inkrementalni algoritam za rješavanje sistema linearnih ograničenja nad realnim brojevima u kontekstu interaktivnih korisničkih interfejsa [3]. Algoritam je razvijen od strane Greg Badros-a, Alan Borning-a i Petera Stuckeya 1998. godine na Univerzitetu Washington, a originalna implementacija u C++ je u kasnijim godinama portovana na gotovo sve popularne programske jezike.

Cassowary rješava problem koji se može formulirati kao:

$$\begin{aligned} \text{minimiziraj} \quad & \sum_i w_i \cdot e_i^2 \\ \text{uz uvjet} \quad & c_j \text{ je zadovoljeno, za sve obavezne (required) } c_j \end{aligned} \tag{1.1}$$

gdje je značenje simbola sljedeće:

- $e_i \geq 0$ – varijabla greške i-tog ne-obaveznog ograničenja; ako je ograničenje zadovoljeno, tada $e_i = 0$
- w_i – numerički prioritet i-tog ograničenja; ograničenja višeg prioriteta dominiraju u minimizaciji
- Obavezna ograničenja c_j (REQUIRED) su tvrdi uvjeti – moraju biti zadovoljena bez izuzetka

Intuitivno: solver minimizira ukupnu teženu sumu narušavanja ne-obaveznih ograničenja, pri čemu obavezna ograničenja moraju uvijek biti potpuno zadovoljena. Ovaj pristup – *hijerarhija ograničenja* – je ključna inovacija Cassowary algoritma u odnosu na klasične LP solvere.

1.2.1 Hijerarhija prioriteta ograničenja

Ključna karakteristika Cassowary algoritma koja ga čini prikladnim za GUI aplikacije je podrška za hijerarhiju prioriteta ograničenja. Umjesto da se svako ograničenje mora bezuvjetno zadovoljiti, korisnik može specificirati koliko je važno neko ograničenje.

Postoje tri korisnički dostupna nivoa prioriteta u Layout Algebra DSL-u:

- **Required** (obavezno) – ograničenje se mora bezuvjetno zadovoljiti; ako nije moguće, solver prijavljuje grešku o konfliktu
- **Strong** (jako) – solver će se potruditi da zadovolji ograničenje, ali će u slučaju konflikta popustiti u korist obaveznih ograničenja
- **Weak** (slabo) – ograničenje koje se čuva samo ako ne postoje konflikti sa svim ostalim ograničenjima viših prioriteta; pogodno za definisanje preferiranih (default) pozicija

Interno, kiwi.js biblioteka koristi i **Medium** nivo prioriteta, koji se u Layout Algebra sistemu koristi isključivo za implementaciju prevlačenja elemenata (edit varijable) i nije direktno dostupan u DSL sintaksi.

Primjer koji ilustruje korisnost ovog koncepta: korisnik definira REQUIRED ograničenje da element ne smije izaći van okvira kontejnera, STRONG ograničenje da element treba biti centriran, i WEAK ograničenje koje definiše željenu veličinu. Solver će automatski pronaći poziciju koja zadovoljava sve prioritete u ispravnom redoslijedu.

1.2.2 Simpleks metoda kao osnova

Cassowary algoritam je zasnovan na **dvostrukoj simpleks metodi** (dual simplex method) linearne optimizacije. Osnovna simpleks metoda rješava problem oblika:

$$\begin{aligned} \text{minimiziraj} \quad & \mathbf{c}^T\mathbf{x} \\ \text{uz uvjet} \quad & A\mathbf{x} = \mathbf{b}, \quad \mathbf{x} \geq 0 \end{aligned} \tag{1.2}$$

gdje je $\mathbf{x} \in \mathbb{R}^n$ vektor varijabli, $\mathbf{c} \in \mathbb{R}^n$ vektor koeficijenata cilja, $A \in \mathbb{R}^{m \times n}$ matrica ograničenja, a $\mathbf{b} \in \mathbb{R}^m$ vektor desnih strana.

Ono što Cassowary dodaje na ovu osnovu je mehanizam za **inkrementalno ažuriranje** – kada se doda ili ukloni ograničenje, ili kada se promijeni vrijednost neke varijable, solver ne mora ponovo rješavati problem od početka. Umjesto toga, ažurira se samo relevantni dio simpleks tablice, što rezultira znatno boljim performansama za interaktivne primjene u kojima se ograničenja često mijenjaju.

1.2.3 Edit varijable i mehanizam sugestija

Za potrebe interaktivnih GUI aplikacija, Cassowary uvodi koncept **edit varijabli**. Ovo je mehanizam koji omogućava korisničkom interfejsu da “sugeriše” željenu vrijednost neke varijable (npr. novu poziciju elementa koji se prevlači), a solver će pronaći najbliže rješenje koje zadovoljava sva ograničenja višeg prioriteta.

Protokol za edit varijable sastoji se od tri koraka:

1. `addEditVariable(var, priority)` – registruje varijablu kao editabilnu s datim prioritetom
2. `suggestValue(var, value)` – sugerira vrijednost za registrovanu edit varijablu
3. `removeEditVariable(var)` – de-registruje varijablu po završetku interakcije

Ovaj mehanizam je direktno iskorišten u implementaciji prevlačenja elemenata u Layout Algebra sistemu, što će biti detaljno opisano u poglavlju o realizaciji.

1.2.4 Kiwi.js biblioteka

U implementaciji ovog rada koristi se **kiwi.js** – JavaScript/TypeScript port Cassowary algoritma [5]. Biblioteka nudi API koji uključuje:

- `kiwi.Variable` – reprezentacija jedne varijable u sistemu
- `kiwi.Expression` – linearna kombinacija varijabli i konstanti
- `kiwi.Constraint` – jedno ograničenje s definisanom jakši
- `kiwi.Solver` – centralna klasa koja drži i rješava sistem
- `kiwi.Strength` – numerički nivoi prioriteta

Sledeći primjer pokazuje osnovno korištenje kiwi.js API-a:

Listing 1.1: Osnovno koristenje kiwi.js solvera

```
import * as kiwi from 'kiwi.js';

const solver = new kiwi.Solver();

// Kreiranje varijabli
const leftA  = new kiwi.Variable('leftA');
const widthA = new kiwi.Variable('widthA');
const leftB  = new kiwi.Variable('leftB');

// Dodavanje ogranicenja: B pocinje gdje A zavrsava + 10px razmaka
const expr = new kiwi.Expression(
  [1, leftA], [1, widthA], 10
);
solver.addConstraint(
  new kiwi.Constraint(
    expr,
    kiwi.Operator.Eq,
    new kiwi.Expression(leftB),
    kiwi.Strength.required
  )
);

// Rjesavanje sistema
solver.updateVariables();
console.log(leftB.value()); // pozicija elementa B
```

Slika 1.1: Tok rješavanja ograničenja – od AST do izračunatih pozicija putem Cassowary/kiwi.js algoritma

1.3 Domenski-specifični jezici (DSL)

Domenski-specifični jezik (Domain-Specific Language, DSL) je programski jezik dizajniran za specifičan problemski domen [6]. Za razliku od jezika opće namjene (General Purpose Languages, GPL) kao što su Java, Python ili TypeScript, DSL-ovi su ograničeni u svom opsegu, ali su zato znatno ekspresivniji i čitljiviji unutar tog opsega.

Klasični primjeri DSL-ova su SQL za upite nad bazama podataka, HTML za strukturiranje web sadržaja, CSS za stilizovanje, te regex za opis regularnih jezika. Svaki od ovih jezika je specijaliziran za svoju domenu i nudi apstrakcije koje su prirodne za tu domenu.

Definicija 1.2 (Eksterni i interni DSL)**.** ***Interni DSL*** *(ugnijezdeni DSL) implementiran je unutar host jezika i koristi njegovu sintaksu uz posebne konvencije.* ***Eksterni DSL*** *ima vlastitu, neovisnu sintaksu i zahtijeva zasebni parser koji prevodi program u form razumljiv sistemu.*

Za potrebe ovog rada implementiran je **eksterni DSL** s imperativnom sintaksom gdje svaka linija počinje ključnom riječju koja opisuje vrstu naredbe.

1.3.1 Principi dizajna dobrog DSL-a

Pri projektovanju DSL-a treba voditi računa o više principa koji osiguravaju dugoročnu upotrebljivost jezika. Fowler [6] navodi sljedeće ključne principe:

- **Čitljivost** – DSL program trebao bi biti razumljiv i osobama koje nisu programeri, bar u grubim crtama. Za layout DSL to znači da ograničenja trebaju ličiti na matematičke jednakosti.
- **Minimalnost** – jezik treba imati što manji skup konstrukata, ali dovoljan za pokrivanje ključnih scenarija. Svaki nepotrebni konstrukt povećava složenost parsera i krivulju učenja.
- **Proširljivost** – dizajn treba dopustiti kasniji dodavanje novih komandi bez narušavanja kompatibilnosti s postojećim programima.
- **Lokalno rezonovanje** – značenje svake linije trebalo bi biti razumljivo iz lokalne okoline, bez potrebe za pravilima globalnog konteksta.

1.3.2 DSL u kontekstu upravljanja layoutom

Layout Algebra DSL je dizajniran da bude:

- **Koncizan** – svaka naredba se piše u jednoj liniji

- **Čitljiv** – sintaksa je bliska matematičkoj notaciji
- **Proširljiv** – lako se dodaju nove komande
- **Dovoljno ekspresivan** – pokriva ključne scenarije rasporeda za web stranice

Centralna ideja je da korisnik može napisati, npr.:

Listing 1.2: Primjer CONSTRAINT naredbi

```
CONSTRAINT card.centerX == container.centerX
CONSTRAINT button.top == card.bottom + 20
```

...i sistem automatski pronađe pozicije koje zadovoljavaju te odnose, bez eksplicitnih koordinata.

1.3.3 Poređenje s alternativnim DSL pristupima za layout

Vrijedi napomenuti da postoje i drugi DSL-ovi koji adresiraju sličan problem. Apple-ov **Visual Format Language (VFL)** [7] je linearni tekstualni format koji se mogao koristiti za definiranje AutoLayout ograničenja u iOS razvoju:

Listing 1.3: Primjer Visual Format Language sintakse

```
H:|-20-[button(>=50)]-20-|
V:|-30-[button(40)]-|
```

VFL je ASCII-art pristup koji vizualno sugerira prostorni raspored (H: za horizontalni raspored, linije i zagrade za elemente i razmake), što ga čini intuitivnim za jednostavne slučajeve. Međutim, za složenije scenarije s više uvjeta VFL postaje teško čitljiv. Apple je 2017. deprecirao VFL u korist vizualnog definiranja ograničenja u Interface Builderu.

Layout Algebra DSL odabire eksplicitniji matematički pristup koji je izražajniji i lakše proširiv.

1.4 React i moderna web arhitektura

React [8] je JavaScript biblioteka za izgradnju korisničkih interfejsa razvijena od strane Meta. Zasniva se na konceptu komponenti – izoliranih jedinica UI logike koje sintetiziraju HTML izlaz iz stanja i ulaznih parametara.

Dva su temeljna koncepta Reacta na kojima počiva arhitektura Layout Algebra aplikacije:

Deklarativno renderovanje – programer opisuje šta treba biti prikazano (na osnovu trenutnog stanja), a React brini o efikasnom ažuriranju DOM-a samo na mjestima gdje se stanje promijenilo. Ovo je konceptualno slično pristupu ograničenja – umjesto imperativnih instrukcija (čini ovo, onda ono), deklariramo željeni ishod.

Jednosmjerni tok podataka (unidirectional data flow) – stanje teče od roditelja prema djeci komponentama kroz props, dok djeca komuniciraju s roditeljima putem callback funkcija. Ovaj pristup čini aplikaciju predvidivijom i lakšom za debugiranje.

1.4.1 Zustand – upravljanje stanjem

Zustand [9] je minimalistična biblioteka za upravljanje globalnim stanjem u React aplikacijama. Za razliku od kompleksnih rješenja kao što je Redux, Zustand nudi jednostavan API zasnovan na hook-ovima:

Listing 1.4: Osnovna struktura Zustand store-a

```
const useLayoutStore = create<LayoutState>((set, get) => ({
  code: '',
  setCode: (code) => set({ code }),
  ast: null,
  positions: {},
  // ...
}));
```

Korištenjem Zustand-a postignuta je centralizirana pohrana svih relevantnih podataka aplikacije. Pored Zustand-a, razmatralo se korištenje Redux Toolkita i React Context API-a. Redux Toolkit je eliminisan zbog pretjerane verboznosti za ovaj tip aplikacije, dok Context API nije pogodan za često se mijenjajuće stanje poput pozicija elemenata (svaka promjena re-renderuje sve konzumente konteksta).

1.4.2 Vite – alat za izgradnju

Vite [10] je moderni alat za izgradnju web projekata koji koristi ES module za razvoj bez bundle-anja, što rezultira izuzetno brzim pokretanjem development servera i Hot Module Replacement-om.

Konfiguracija za ovaj projekt je minimalna:

Listing 1.5: Vite konfiguracija projekta

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    open: true,
  },
});
```

1.4.3 TypeScript – sistem tipova

TypeScript je nadskup JavaScript-a koji dodaje statičku provjeru tipova. U kontekstu ovog projekta TypeScript je imao više konkretnih prednosti:

Definicije tipova za AST strukture spriječile su čitavu klasu grešaka tokom razvoja parsera i solvera. TypeScript-ov *discriminated union* tip je posebno koristan za modeliranje različitih tipova čvorova AST-a, jer kompajler može provjeriti je li slučaj svake grane pokriven:

Listing 1.6: Primjer discriminated union za DSL komande

```
type DSLCommand =
  | { kind: 'container'; width: number; height: number }
  | { kind: 'element';   id: string; w: number; h: number }
  | { kind: 'constraint'; lhs: PropRef; op: Op; rhs: RHSExpr }
  | { kind: 'color';     id: string; hex: string }
  | { kind: 'parent';    child: string; parent: string }
  | { kind: 'avoid';     a: string; b: string };
```

U ovom poglavlju su izložene teorijske osnove sistema. Cassowary algoritam omogućava efikasno rješavanje ograničenja u realnom vremenu. Eksterna DSL arhitektura nudi čitljiv i koncizan način opisivanja odnosa. React ekosistem s Zustand i Vite alatima pruža solidnu tehničku infrastrukturu. U nastavku slijedi poglavlje koje detaljno opisuje implementaciju.

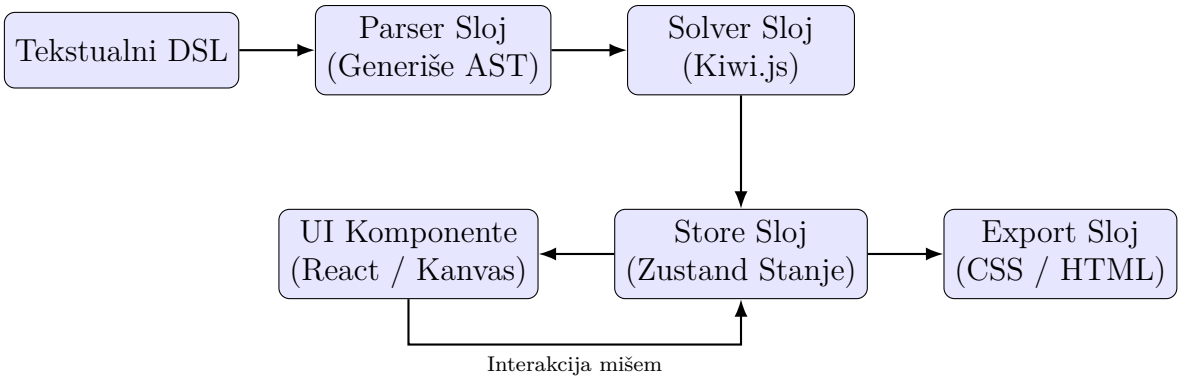
2 Realizacija sistema Layout Algebra

2.1 Arhitektura sistema

Sistem Layout Algebra je organiziran u sljedeće logičke slojeve:

1. **Parser sloj** (`src/parser/`) – pretvara tekstualni DSL u strukturirani AST
2. **Solver sloj** (`src/solver/`) – rješava sistem ograničenja i izračunava pozicije
3. **Store sloj** (`src/store/`) – centralizirano upravljanje stanjem aplikacije
4. **UI komponente** (`src/components/`) – React komponente za vizualizaciju
5. **Export sloj** (`src/export/`) – transformacija riješenog layouta u CSS/HTML

Dijagram toka podataka: korisnik unosi DSL kod → parser generiše AST → solver prima AST i izračunava pozicije → store ažurira stanje → React re-renderuje UI. Obrnuti smjer (interakcija mišem) → store direktno ažurira pozicije → DSL se ažurira po potrebi.



Slika 2.1: Arhitektura sistema Layout Algebra – prikaz slojeva i toka podataka između komponenata

Ovakva slojevita arhitektura osigurava jasno razdvajanje odgovornosti (separation of concerns) – parser ne zna ništa o solverovim internim detaljima, solver ne zna kako se rezultati prikazuju, a UI komponente ne znaju ništa o tome kako su ograničenja riješena. Ova izolacija olakšava testiranje i buduće izmjene pojedinih slojeva.

2.2 DSL parser

Parser je implementiran u modulu `src/parser/parser.ts`. Funkcija `parseDSL(code: string)` prihvata string s DSL kodom i vraća `{ast, error}`.

Pristup koji je odabran je **rekurzivni descent parser** – jedan od najjednostavnijih i najdirektnije implementabilnih tipova parsera. Za jezik koji ima jednostavnu linijsku gramatiku (svaka naredba se počinje prepoznati iz prve ključne riječi), ova metoda je optimalan izbor. Alternativa bi bio generator parsera (npr. ANTLR ili PEG.js), koji bi donio više infrastrukturne kompleksnosti bez značajne dobiti za ovaj opseg problema.

Parsiranje se odvija u dva logička koraka koji su u implementaciji spojeni u jednu prolaz:

Leksička analiza (tokenizacija): svaka linija se dijeli na tokene razdvajanjem po razmacima. S obzirom da Layout Algebra DSL nema složene leksičke konstrukte (npr. višeriječne stringove ili escape sekvence), ova tokenizacija je trivijalna i ne zahtijeva posebni lekser.

Sintaksna analiza (parsiranje pravila): na osnovu prve riječi u liniji (ključne riječi), parser bira odgovarajuću funkciju za parsiranje ostatka linije i kreira odgovarajući čvor u AST-u.

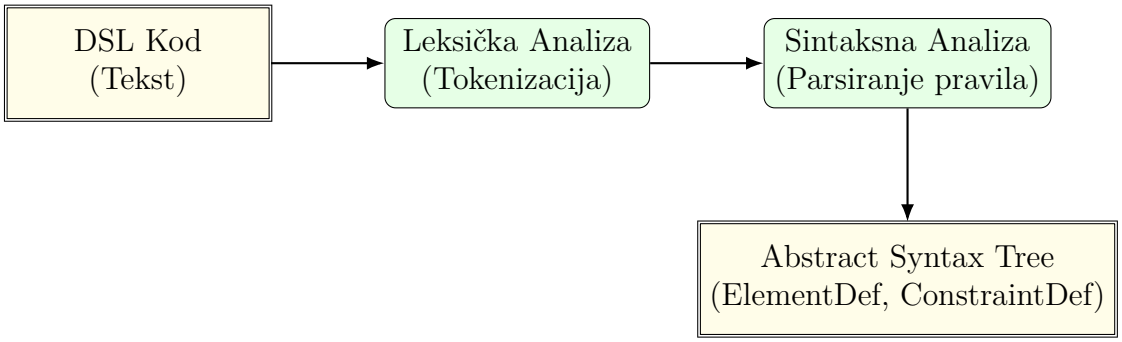
AST (*Abstract Syntax Tree*) je definisan interfejsom:

Listing 2.1: Definicija AST strukture

```
interface AST {
  elements:    ElementDef[];
  constraints: ConstraintDef[];
  colors:      ColorDef[];
  hierarchy:   HierarchyDef[];
  avoids:      AvoidDef[];
}

interface ElementDef {
  id:     string;
  width:  number | string; // broj ili "50%"
  height: number | string;
}

interface ConstraintDef {
  lhs:      PropRef;
  op:       '==' | '>=' | '<=';
  rhs:      RHSValue;
  strength: 'required' | 'strong' | 'weak';
}

interface PropRef {
```

```
  element:  string;
  property: Property;
}
```

Parser radi liniju po liniju i prepoznaje naredbe na osnovu ključne riječi s kojom linija počinje. Komentari (linije koje počinju s `//`) i prazne linije se ignoriraju.



Slika 2.2: Tok parsiranja DSL koda – transformacija tekstualnog ulaza u AST strukturu

2.2.1 Podržane komande

Tablica 2.1: Pregled komandi Layout Algebra DSL

Komanda	Sintaksa	Opis
CONTAINER	`CONTAINER w h`	Dimenzije kontejnera
ELEMENT	`ELEMENT id w h`	Definisanje elementa (fiksno ili %)
CONSTRAINT	`CONSTRAINT l op r [jakost]`	Definisanje ograničenja
COLOR	`COLOR id #hex`	Boja elementa
PARENT	`PARENT dijete roditelj`	Hijerarhija
AVOID	`AVOID id1 id2`	Izbjegavanje preklapanja

Tablica 2.2: Podržane karakteristike elemenata u ograničenjima

Svojstvo	Opis
`left`, `right`	Lijevi/desni rub elementa
`top`, `bottom`	Gornji/donji rub elementa
`width`, `height`	Širina i visina
`centerX`, `centerY`	Horizontalni/vertikalni centar

2.2.2 Parsiranje ograničenja

Parsiranje CONSTRAINT naredbi je najsloženija logika parsera. Podržavaju se dva obrasca:

Obrazac 1 – odnos između svojstava dva elementa (s opcionalnim množiteljem i pomakom):

```
CONSTRAINT a.prop == b.prop [* faktor | / faktor] [+/- pomak] [
    jakost]
```

Obrazac 2 – postavljanje karakteristike elementa na konstantu:

```
CONSTRAINT a.prop == 300 [jakost]
```

Podržani su operatori poređenja ==, >= i <=. Parser ekstraktira opcionalnu jakosnu oznaku (WEAK, STRONG, REQUIRED) i mapira je na odgovarajući numerički prioritet za solver.

Posebno vrijedi napomenuti da parser mora razlikovati referencu na element (oblik `id.property`) od literalne konstante (broj) na desnoj strani ograničenja. Ova razlika je ključna jer dovodi do generisanja različitih kiwi.js izraza u fazi rješavanja.

Sljedeći isječak ilustruje kako parser procesira CONSTRAINT liniju:

Listing 2.2: Pseudokod logike parsiranja CONSTRAINT naredbe

```
function parseConstraint(tokens: string[]): ConstraintDef {
  // tokens = ["CONSTRAINT", "a.centerX", "==", "b.centerX", "+",
    "10", "WEAK"]
  const lhs = parsePropRef(tokens[1]); // { element: 'a', property
    : 'centerX' }
  const op  = tokens[2] as Op;          // '=='

  // Da li je desna strana broj ili referenca?
  const rhsToken = tokens[3];
  let rhs: RHSValue;
  if (!isNaN(Number(rhsToken))) {
    rhs = { kind: 'constant', value: Number(rhsToken) };
  } else {
    rhs = {
      kind:    'propref',
      ref:     parsePropRef(rhsToken),
      factor:  parseOptionalFactor(tokens, 4),
      offset:  parseOptionalOffset(tokens, 5),
    };
  }
  const strength = parseStrength(tokens);
  return { lhs, op, rhs, strength };
}
```

2.2.3 Rukovanje greškama u parseru

Dobar parser ne samo da prepoznaje ispravne programe već i pruža korisne poruke o greškama. U Layout Algebra DSL parseru implementirano je nekoliko vrsta detekcije grešaka:

- **Nepoznata ključna riječ**: ako linija ne počinje prepoznatom ključnom riječi, parser generiše grešku s brojem linije.
- **Neispravna referenca na element**: ako se u CONSTRAINT naredbi referira nepostojeći element (koji nije prethodno definiran ELEMENT naredbom), parser to prijavljuje.
- **Neispravna karakteristika elementa**: ako se koristi ime karakteristike koje nije u skupu {left, right, top, bottom, width, height, centerX, centerY}, parser generiše upozorenje.
- **Neispravan broj argumenata**: svaka naredba očekuje tačno određeni broj argumenata; višak ili manjak argumenata se prijavljuje.

Greške se prikazuju korisniku u UI s jasnom porukom i brojem linije gdje je greška nastala, što znatno olakšava debugiranje DSL koda.

2.3 Solver – rješavač ograničenja

Klasa `LayoutSolver` u modulu `src/solver/solver.ts` enkapsulira kiwi.js solver. Svaki put kada se DSL kod promijeni, poziva se metoda `update(ast, cw, ch)` koja:

1. Gradi mapu hijerarhije (parent tree) iz AST-a
2. Sprema trenutne pozicije za konzistentnost pri promjeni hijerarhije
3. Kreira novi Solver i čisti sve varijable
4. Za svaki element kreira 8 varijabli: `left, right, top, bottom, width, height, centerX, centerY`, te `localLeft` i `localTop`
5. Dodaje strukturalna ograničenja (veze između varijabli)
6. Procesira hijerarhijske odnose: globalna pozicija djeteta = pozicija roditelja + lokalna pozicija
7. Dodaje DSL ograničenja s odgovarajućim prioritetima
8. Poziva `solver.updateVariables()`

2.3.1 Strukturalna ograničenja

Za svaki element, pored DSL ograničenja koja definira korisnik, solver dodaje i skup **strukturalnih ograničenja** koja održavaju koherenciju varijabli elementa. Ova ograničenja su uvijek na nivou REQUIRED i korisnik ih ne može nadvladati:

$$\begin{aligned}
\texttt{right} &= \texttt{left} + \texttt{width} \\
\texttt{bottom} &= \texttt{top} + \texttt{height} \\
\texttt{centerX} &= \texttt{left} + \texttt{width}/2 \\
\texttt{centerY} &= \texttt{top} + \texttt{height}/2 \\
\texttt{width} &\geq 0 \\
\texttt{height} &\geq 0
\end{aligned}$$

Ova ograničenja su temelj konzistentnosti: zahvaljujući njima, korisnik može koristiti bilo koje od 8 svojstava u CONSTRAINT naredbama i solver će uvijek održavati pravilne odnose između njih.

2.3.2 Generisanje kiwi.js ograničenja iz AST-a

Središnji dio solvera jeste transformacija `ConstraintDef` objekata iz AST-a u odgovarajuće `kiwi.Constraint` objekte. Ova transformacija mora pokriva oba oblika

Tablica 2.3: Nivoi prioriteta ograničenja

Naziv	Numerička vrijednost	Značenje
REQUIRED	1 001 001 000	Mora biti zadovoljeno
STRONG	1 000 000	Jako poželjno
MEDIUM	1 000	Srednje poželjno
WEAK	1	Slabo poželjno

desne strane (konstantu i referencu na karakteristiku drugog elementa), te opcionalnu multiplikaciju i pomak.

Listing 2.3: Logika generisanja kiwi.js ogranicenja

```
function buildKiwiConstraint(
  def: ConstraintDef,
  vars: Map<string, ElementVars>
): kiwi.Constraint {
  const lhsVar = getVar(vars, def.lhs);

  let rhsExpr: kiwi.Expression;
  if (def.rhs.kind === 'constant') {
    rhsExpr = new kiwi.Expression(def.rhs.value);
  } else {
    const rhsVar = getVar(vars, def.rhs.ref);
    const factor = def.rhs.factor ?? 1;
    const offset = def.rhs.offset ?? 0;
    rhsExpr = new kiwi.Expression([factor, rhsVar], offset);
  }

  const strength = strengthMap[def.strength];
  return new kiwi.Constraint(
    new kiwi.Expression(lhsVar),
    operatorMap[def.op],
    rhsExpr,
    strength
  );
}
```

2.3.3 Implementacija prevlačenja

Prevlačenje elemenata zahtijeva poseban tretman putem *edit varijabli*. Implementacija je podijeljena u tri faze koje odgovaraju životnom ciklusu jedne operacije prevlačenja:

1. `beginDrag(id)` – dodaje `left` i `top` varijable elementa kao edit varijable s prioritetom `medium`; roditelji se zaključavaju s `strong`

2. `drag(id, x, y)` – poziva `suggestValue` i `updateVariables`; solver pronalazi stanje koje zadovoljava sugestiju bez narušavanja viših ograničenja
3. `endDrag(id)` – uklanja edit varijable i sprema finalne lokalne koordinate

Ovaj dizajn ima jednu važnu karakteristiku: tokom prevlačenja, solver i dalje poštuje sva REQUIRED i STRONG ograničenja. Dakle, ako korisnik pokuša prevući element izvan kontejnera, a postoji REQUIRED ograničenje koje to zabranjuje, element će se zaustaviti na granici – baš kao što bi AutoLayout to napravio u iOS okruženju.

2.3.4 Implementacija AVOID pravila

AVOID pravilo je implementirano kao skup četiri slaba ograničenja koja zahtijevaju da se dva elementa ne preklapaju u barem jednoj dimenziji. Preciznije, za dva elementa A i B, AVOID generira ograničenja:

$$
\begin{aligned}
A.\texttt{right} &\leq B.\texttt{left} \quad \text{(WEAK)} \\
B.\texttt{right} &\leq A.\texttt{left} \quad \text{(WEAK)} \\
A.\texttt{bottom} &\leq B.\texttt{top} \quad \text{(WEAK)} \\
B.\texttt{bottom} &\leq A.\texttt{top} \quad \text{(WEAK)}
\end{aligned}
$$

Solver će zadovoljiti bar jedno od ovih ograničenja na osnovu trenutnih pozicija elemenata, efektivno razdvajajući ih u smjeru koji zahtijeva manji pomak. Ovo je aproksimacija pravog ne-preklapajućeg ograničenja (koje bi zahtijevalo nelinearno programiranje), ali funkcionira zadovoljavajuće u većini praktičnih slučajeva.

2.4 Upravljanje stanjem aplikacije

Centralno upravljanje stanjem realizirano je u `src/store/layoutStore.ts` putem Zustand store-a. Store drži sljedeće podatke:

- `code` – trenutni DSL kod
- `ast` – parsed AST ili `null`
- `error` – poruka greške ili `null`
- `positions` – izračunate pozicije svih elemenata
- `positionOverrides` – manualno postavljene pozicije/dimenzije
- `solver` – instanca `LayoutSolver` klase
- `groups` – lista grupiranih parova
- `history`, `historyIndex` – undo/redo stanja

2.4.1 Reaktivni tok podataka

Važan aspekt arhitekture store-a jeste reaktivnost – svaka promjena DSL koda automatski pokreće niz operacija koje kulminiraju ažuriranjem UI-a. Sljedeći pseudokod ilustrira ovaj tok:

Listing 2.4: Reaktivni tok od promjene koda do azuriranja UI-a

```
const setCode = (newCode: string) => {
  const { ast, error } = parseDSL(newCode);
  let positions = get().positions;

  if (ast && !error) {
    const solver = get().solver;
    positions = solver.update(ast, containerW, containerH);
  }

  set({ code: newCode, ast, error, positions });
  // React ce automatski re-renderovati sve komponente
  // koje koriste 'positions' iz store-a
};
```

2.4.2 Undo/Redo mehanizam

Undo/Redo je implementiran kao linearni niz `HistoryEntry` objekata s maksimumom od 50 unosa:

Listing 2.5: Struktura historijskog unosa

```
interface HistoryEntry {
  code:              string;
```

```
  positionOverrides: Record<string, Rect>;
  groups:            GroupEntry[];
}
```

Metoda `pushHistory()` dodaje trenutno stanje na historiju. `undo()` dekrementira index i restaurira prethodno stanje, a `redo()` inkrementira index prema naprijed.

Implementacija koristi **linearni model historije**: svaka nova akcija briše sve unose koji se nalaze ispred trenutnog indeksa (tzv. „grana koja se ne može ponovo prisećati”). Ovo je standardni pristup koji koriste editori teksta, Photoshop i većina profesionalnih alata.

Ograničenje od 50 koraka je odabrano kao kompromis između memorijskog otiska i praktične korisnosti. Za većinu interaktivnih sesija 50 koraka je više nego dovoljno, a buduća implementacija bi mogla proširiti ovu granicu korištenjem kompresije ili čuvanjem samo diferencijalnih izmjena (delta compression).

2.4.3 Grupisanje elemenata

Grupisanje je mehanizam kojim se prevlačenjem jednog elementa pomjeraju i drugi, uz očuvanje relativnog razmaka. Svaka grupa se pohranjuje kao:

Listing 2.6: Struktura podataka za grupisanje

```
interface GroupEntry {
  leaderId:   string;  // ID vodeceg elementa
  followerId: string;  // ID pratioce elementa
  offsetX:    number;  // horizontalni relativni offset
  offsetY:    number;  // vertikalni relativni offset
  gapX:       number;  // horizontalni razmak izmedju rubova
  gapY:       number;  // vertikalni razmak izmedju rubova
}
```

Grupno prevlačenje je implementirano rekurzivno u metodi `handleDrag`, s klampovanjem unutar roditeljskih granica. Rekurzija je potrebna jer follower može i sam biti leader nekog drugog elementa, čime se formiraju *lanci grupisanja* koji se moraju propagirati u pravom redoslijedu.

2.5 Korisnički interfejs

Korisnički interfejs aplikacije organiziran je u tri glavne vertikalne cjeline koje zajedno čine radni prostor sistema: lijevi panel za uređivanje i alate, centralni vizualni kanvas te desni panel sa svojstvima elemenata.

Lijevi panel sastoji se od dvije osnovne sekcije:

- *DSL Editor* – tekstualni editor u kojem korisnik definiše layout koristeći DSL sintaksu. Editor prikazuje definicije kontejnera, elemenata, boja i hijerarhijskih odnosa između elemenata.
- *Constraint Builder* – grafički alat za kreiranje ograničenja izborom elemenata, njihovih osobina i relacijskih operatora, bez potrebe za ručnim pisanjem DSL koda.

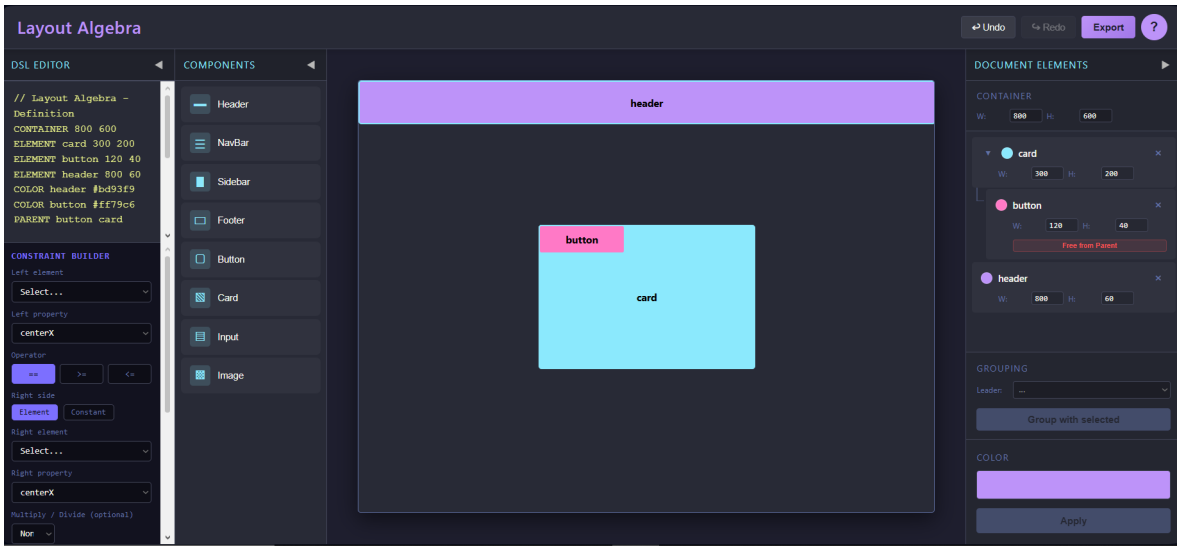
Pored editora nalazi se i sekcija *Components*, koja sadrži listu predefiniranih UI komponenti (Header, NavBar, Sidebar, Footer, Button, Card, Input i Image) koje je moguće dodati u layout.

Centralni dio interfejsa predstavlja *vizualni kanvas*, odnosno radnu površinu na kojoj se prikazuje raspored elemenata izračunat pomoću constraint solvera. Elementi su vizualno predstavljeni pravougaonicima različitih boja i oznaka (npr. header, card, button), a njihov položaj i veličina odgovaraju DSL definiciji.

Desni panel služi za upravljanje svojstvima selektovanih elemenata i sadrži:

- *Document Elements* – listu svih elemenata unutar kontejnera zajedno sa njihovim dimenzijama (širina i visina).
- *Grouping* – opcije za grupisanje elemenata i definisanje hijerarhijskih odnosa.
- *Color* – alat za promjenu boje trenutno selektovanog elementa.

Na vrhu aplikacije nalazi se *toolbar* sa komandama za Undo/Redo akcije, export layouta i dodatne opcije pomoći.



Slika 2.3: Korisnički interfejs aplikacije Layout Algebra sa DSL editorom, komponentama, vizualnim kanvasom i panelom osobina

2.5.1 Vizualni kanvas

Komponenta `Canvas.tsx` prikazuje sve elemente kao apsolutno pozicionirane `div` elemente. Svaki element prikazuje svoju oznaku te 8 resize handles za promjenu veličine (jedan na svakom uglu i jedan na sredini svake stranice).

Prevlačenje je implementirano putem globalnih `mousemove`/`mouseup` event listenera koji se dodaju na `document` objekt tokom aktive drag operacije. Ovim se izbjegava problem „iscurivanja” miša izvan elementa tokom brzog kretanja.

Z-index je izračunat na osnovu dubine u hijerarhiji i indeksa posljednje selekcije:

$$z = \text{selectionBoost} \times 100 + \text{depth} \times 10 + 1$$

gdje `selectionBoost` $= 1$ ako je element trenutno selektiran (inace 0), a `depth` je dubina u hijerarhiji roditelj-dijete (korijen ima dubinu 0). Ovim se osigurava da selektiran element uvijek bude na vrhu, ali da djeca uvijek budu nacrtana iznad roditelja.

2.5.2 DSL editor i Constraint Builder

DSL editor je `<textarea>` element koji prikazuje i prima DSL kod. Svaka promjena koda odmah pokreće parsing i rješavanje. Editor je namjerno implementiran bez naprednih funkcija (kao što su syntax highlighting ili autocomplete) kako bi se izbjegla ovisnost o teškim editor bibliotekama. Ove funkcije su predviđene za buduće verzije sistema.

Constraint Builder (`ConstraintBuilder.tsx`) je grafički alat koji omogućava kreiranje ograničenja bez pisanja DSL koda ručno. Korisniku nudi:

- Dropdown izbor elemenata i osobina (lijeva i desna strana)
- Odabir operatora (==, >=, <=)
- Opcionalnog množitelja/djelitelja i pomaka
- Odabir prioriteta (REQUIRED / STRONG / WEAK)
- Live preview generisane CONSTRAINT linije

Kada korisnik klikne „Dodaj ograničenje”, Constraint Builder generira odgovarajuću CONSTRAINT liniju i dodaje je na kraj DSL koda. Ovo je korisno za korisnike koji još nisu upoznati s DSL sintaksom, ali žele kreirati ograničenja.

2.5.3 Panel elemenata i hijerarhija

Komponenta `ElementListPanel.tsx` prikazuje listu svih elemenata u AST-u i nudi: selekciju, brisanje, promjenu boje, promjenu roditelja i upravljanje grupama.

Hijerarhijski odnosi roditelj-dijete prikazani su uvlačenjem (indentacijom) u listi. Promjena roditelja elementa se vrši putem dropdown menija – kada korisnik odabere novog roditelja, aplikacija automatski dodaje ili mijenja odgovarajuću PARENT naredbu u DSL kodu i odmah reizračunava layout.

2.6 Export modul

Modul `src/export/cssExport.ts` implementira konverziju riješenog layouta u standardne web formate. Ulaz u ovaj modul je mapa izračunatih pozicija (rezultat solvera), a izlaz je string s CSS ili HTML kodom.

Tablica 2.4: Podržani formati eksporta

Format	Opis
Absolute CSS	`position: absolute` s px vrijednostima ili postotcima
Flexbox CSS	Aproksimacija layouta putem CSS Flexbox
CSS Grid	Tačan prikaz putem CSS Grid-a s template definicijom
HTML File	Kompletan HTML fajl s ugrađenim `<style>` blokom

2.6.1 Apsolutno pozicioniranje

Ovo je najdirektnije preslikavanje riješenog layouta u CSS. Za svaki element generiše se:

Listing 2.7: Primjer generisanog apsolutnog CSS-a

```
.container {
  position: relative;
  width: 800px;
  height: 600px;
}
.card {
  position: absolute;
  left: 250px;
  top: 200px;
  width: 300px;
  height: 200px;
}
```

Apsolutno pozicioniranje je idealno za jednokratne, statičke layoute gdje se ne očekuje promjena veličine prozora. Za elemente s postotnim dimenzijama u DSL-u, export koristi postotke umjesto piksela gdje je to moguće.

2.6.2 CSS Grid eksport

Grid eksport je općenito precizniji od Flexbox eksporta jer CSS Grid nudi dvodimenzionalnu kontrolu pozicioniranja. Algoritam za generisanje Grid eksporta radi na sljedeći način:

1. Prikupljaju se sve jedinstvene x-koordinate lijevih i desnih rubova svih elemenata, te sve jedinstvene y-koordinate gornjih i donjih rubova.
2. Te koordinate definiraju kolone i redove Grid-a (template).
3. Za svaki element računa se koja grid kolona/red mu odgovara i generiše se `grid-column` i `grid-row` deklaracija.

2.6.3 Flexbox eksport

Flexbox eksport je najsloženiji od četiri formata, jer Flexbox je jednodimenzionalni layout model koji ne odgovara direktno apstraktnim koordinatama. Implementacija analizira raspored elemenata i pokušava prepoznati obrasce koji se mogu aproksimirati Flexbox modelom:

- Elementi poredani horizontalno s približno istim `top` vrijednostima – mapiraju se na `flex-direction: row`
- Elementi poredani vertikalno s približno istim `left` vrijednostima – mapiraju se na `flex-direction: column`
- Ostali slučajevi – aproksimiraju se pomoću `position: absolute` unutar flex kontejnera

Flexbox eksport će uvijek biti aproksimacija, jer postoje raspored koji se ne mogu precizno reprezentovati Flexbox modelom bez promjene strukture HTML-a. Ovo ograničenje je dokumentirano u korisničkom interfejsu – korisnik je obaviješten da Flexbox export možda neće biti savršeno precizan.

U ovom poglavlju je opisana implementacija svih komponenata sistema. Svaki sloj ima jasno definisanu odgovornost. U nastavku se analiziraju konkretni rezultati upotrebe i ograničenja rješenja.

3 Rezultati i analiza

3.1 Primjeri upotrebe

U ovom poglavlju su prikazana četiri karakteristična primjera koji demonstriraju različite aspekte Layout Algebra sistema. Svaki primjer je odabran da ilustrira specifičan scenario koji se često pojavljuje u praksi web dizajna, a koji je problematičan ili neprirodan za rješavanje putem klasičnih CSS pristupa.

3.1.1 Primjer 1: Dashboard layout

Dashboard layouti su jedan od najčešćih i najsloženijih scenarija u web dizajnu. Sastoje se od više međuzavisnih panela čiji se položaji moraju održavati i nakon promjene dimenzija bilo kojeg elementa. Sljedeći primjer demonstrira topbar, sidebar, tri metrik-kartice u redu, grafikon i tabelu – sve opisano ograničenjima bez eksplicitnih koordinata.

Listing 3.1: DSL kod za Primjer 1 – Dashboard layout

```
CONTAINER 1200 800
ELEMENT topbar    1200 56
ELEMENT sidebar    240 744
ELEMENT card1      320 180
ELEMENT card2      320 180
ELEMENT card3      320 180
ELEMENT chart      680 360
ELEMENT tablePanel 680 280

COLOR topbar    #282a36
COLOR sidebar   #44475a
COLOR card1     #6272a4
COLOR card2     #bd93f9
COLOR card3     #ff79c6
COLOR chart     #f8f8f2
COLOR tablePanel #44475a

CONSTRAINT topbar.top    == container.top
CONSTRAINT topbar.left   == container.left
CONSTRAINT sidebar.top   == topbar.bottom
CONSTRAINT sidebar.left  == container.left

// Tri kartice u redu, automatski poravnanje
CONSTRAINT card1.top     == topbar.bottom + 20
```

```
CONSTRAINT card1.left     == sidebar.right + 20
CONSTRAINT card2.top      == card1.top
CONSTRAINT card2.left     == card1.right + 20
CONSTRAINT card3.top      == card1.top
CONSTRAINT card3.left     == card2.right + 20

// Grafikon ispod kartica
CONSTRAINT chart.top      == card1.bottom + 20
CONSTRAINT chart.left     == sidebar.right + 20

// Tabela pored grafikona
CONSTRAINT tablePanel.top    == chart.top
CONSTRAINT tablePanel.left   == chart.right + 20
CONSTRAINT tablePanel.right == container.right - 20
```

Solver automatski održava poravnanje triju kartica (‘card1‘, ‘card2‘, ‘card3‘) zahvaljujući ograničenjima `card2.top == card1.top` i `card3.top == card1.top`. Ako se promijeni visina topbara, cijeli ostatak layouta automatski klizi prema dolje – niti jedna apsolutna koordinata nije ručno zadana.

Slika 3.1: Rezultat Primjera 1 – dashboard layout s topbarom, sidebarom, tri kartice u redu, grafikonom i tabelom

3.1.2 Primjer 2: Forma s postotnim dimenzijama i ugniježdenim elementima

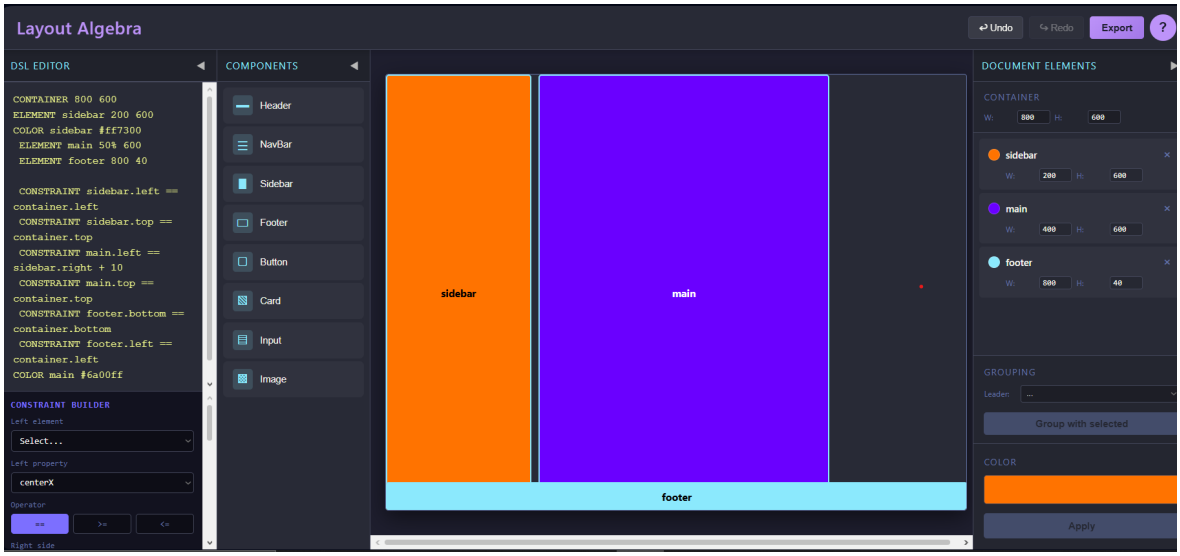
Ovaj primjer ilustrira miješanje apsolutnih i relativnih (%) dimenzija, te dublje ugnježdavanje elemenata unutar roditeljskog kontejnera. Forma zauzima 60% širine kontejnera, a svi unutarnji elementi poravnati su centralno unutar nje bez eksplicitnih koordinata.

Listing 3.2: DSL kod za Primjer 2 – Forma s postotnim dimenzijama

```
CONTAINER 900 700
ELEMENT formCard    60%  500
ELEMENT titleLabel  80%   36
ELEMENT inputName   80%   44
ELEMENT inputEmail  80%   44
ELEMENT inputMsg    80%  120
ELEMENT btnSubmit   40%   44
ELEMENT btnCancel   40%   44
ELEMENT infoPanel   30%  500

COLOR formCard   #282a36
COLOR titleLabel #bd93f9
COLOR inputName  #44475a
COLOR inputEmail #44475a
COLOR inputMsg   #44475a
COLOR btnSubmit  #50fa7b
COLOR btnCancel  #ff5555
COLOR infoPanel  #44475a

PARENT titleLabel formCard
PARENT inputName  formCard
PARENT inputEmail formCard
PARENT inputMsg   formCard
PARENT btnSubmit  formCard
PARENT btnCancel  formCard

CONSTRAINT formCard.top    == container.top  + 30
CONSTRAINT formCard.left   == container.left + 20

// Elementi unutar forme centralno poravnati
CONSTRAINT titleLabel.centerX == formCard.centerX
CONSTRAINT titleLabel.top     == formCard.top + 20

CONSTRAINT inputName.centerX  == formCard.centerX
CONSTRAINT inputName.top      == titleLabel.bottom + 24

CONSTRAINT inputEmail.centerX == formCard.centerX
CONSTRAINT inputEmail.top     == inputName.bottom + 12

CONSTRAINT inputMsg.centerX   == formCard.centerX
CONSTRAINT inputMsg.top       == inputEmail.bottom + 12

// Dva dugmeta jedno pored drugog
CONSTRAINT btnSubmit.top   == inputMsg.bottom + 20
CONSTRAINT btnSubmit.left  == inputName.left
CONSTRAINT btnCancel.top   == btnSubmit.top
```

```
CONSTRAINT btnCancel.left   == btnSubmit.right + 12

// Info panel uz desni rub kontejnera
CONSTRAINT infoPanel.top    == formCard.top
CONSTRAINT infoPanel.right  == container.right - 20
```

Solver izračunava širinu forme kao 60% × 900 = 540 px, a sve `centerX` veze automatski određuju horizontalnu poziciju svakog input polja bez ijedne ručno zadane x-koordinate. Ako se promijeni širina kontejnera, cijela forma se proporcionalno skalira.



Slika 3.2: Rezultat Primjera 2 – forma s postotnim dimenzijama, ugniježdenim elementima i info panelom

3.1.3 Primjer 3: Izbjegavanje preklapanja s hijerarhijom prioriteta

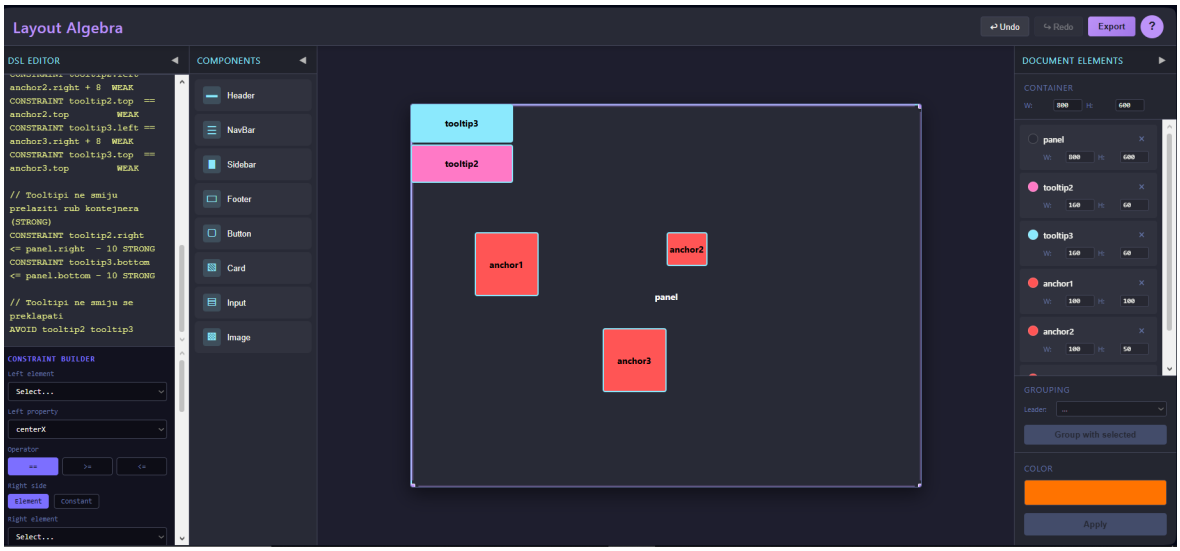
Ovaj primjer demonstrira AVOID pravilo u kombinaciji s hijerarhijom prioriteta ograničenja. Tri tooltipa pokušavaju se pozicionirati pored svojih sidra (WEAK), ali moraju ostati unutar granica kontejnera (STRONG) i ne smiju se međusobno preklapati (AVOID).

Listing 3.3: DSL kod za Primjer 3 – AVOID s prioritetima

```
//{DSL kod za Primjer 3 -- Kolizija i prioriteti ogranichenja}]
CONTAINER 800 600
ELEMENT panel    800 600
ELEMENT tooltip2 160  60
ELEMENT tooltip3 160  60
ELEMENT anchor1   100  100
ELEMENT anchor2 100 50
ELEMENT anchor3   100  100

```

```
COLOR panel    #282a36
COLOR tooltip2 #ff79c6
COLOR tooltip3 #8be9fd
COLOR anchor1  #ff5555
COLOR anchor2  #ff5555
COLOR anchor3  #ff5555

// Sidra na fiksnim pozicijama
CONSTRAINT anchor1.left == 100
CONSTRAINT anchor1.top  == 200
CONSTRAINT anchor2.left == 400
CONSTRAINT anchor2.top  == 200
CONSTRAINT anchor3.left == 300
CONSTRAINT anchor3.top  == 350

// Tooltipi se pokusavaju grupisati blizu sidra (WEAK)
CONSTRAINT tooltip2.left == anchor2.right + 8  WEAK
CONSTRAINT tooltip2.top  == anchor2.top         WEAK
CONSTRAINT tooltip3.left == anchor3.right + 8  WEAK
CONSTRAINT tooltip3.top  == anchor3.top         WEAK

// Tooltipi ne smiju prelaziti rub kontejnera (STRONG)
CONSTRAINT tooltip2.right  <= panel.right  - 10 STRONG
CONSTRAINT tooltip3.bottom <= panel.bottom - 10 STRONG

// Tooltipi ne smiju se preklapati
AVOID tooltip2 tooltip3
```

Hijerarhija prioriteta ovdje je ključna: solver će najprije zadovoljiti REQUIRED strukturalna ograničenja, zatim STRONG ograničenje ostanka unutar kontejnera, a tek onda WEAK ograničenje blizine sidru. Ako je tooltip blizu desnog ruba, solver ga automatski gura prema lijevo dok ne zadovolji STRONG uvjet – bez ikakvog eksplicitnog koda za detekciju kolizije.



Slika 3.3: Rezultat Primjera 3 – tri tooltipa s AVOID pravilom i hijerarhijom prioriteta; solver automatski rješava kolizije i granice kontejnera

3.1.4 Primjer 4: Višerazinska hijerarhija – landing page

Ovaj primjer demonstrira najsloženiju funkcionalnost sistema: višerazinsku hijerarhiju roditelj-dijete u kojoj su elementi unutar elemenata unutar kontejnera, s ukupno 28 ograničenja koja se rješavaju istovremeno.

Listing 3.4: DSL kod za Primjer 4 – Višerazinska hijerarhija

```
CONTAINER 1024 768
ELEMENT navbar      1024  64
ELEMENT logo         120  40
ELEMENT navBtn1       80  36
ELEMENT navBtn2       80  36
ELEMENT hero        1024 300
ELEMENT heroTitle    480  56
ELEMENT heroSub      380  36
ELEMENT heroCta      160  48
ELEMENT gridRow      960 220
ELEMENT featureA     280 200
ELEMENT featureB     280 200
ELEMENT featureC     280 200
ELEMENT iconA         48  48
ELEMENT iconB         48  48
ELEMENT iconC         48  48

COLOR navbar    #282a36
COLOR hero      #44475a
COLOR heroTitle #f8f8f2
COLOR heroCta   #50fa7b
COLOR gridRow   #282a36
```

```
23 COLOR featureA  #6272a4
24 COLOR featureB  #bd93f9
25 COLOR featureC  #ff79c6
26 COLOR iconA     #ffb86c
27 COLOR iconB     #ffb86c
28 COLOR iconC     #ffb86c
29
30 // Hijerarhija -- tri nivoa gnijezdjenja
31 PARENT logo     navbar
32 PARENT navBtn1  navbar
33 PARENT navBtn2  navbar
34 PARENT heroTitle hero
35 PARENT heroSub   hero
36 PARENT heroCta   hero
37 PARENT featureA  gridRow
38 PARENT featureB  gridRow
39 PARENT featureC  gridRow
40 PARENT iconA     featureA
41 PARENT iconB     featureB
42 PARENT iconC     featureC
43
44 // Navbar
45 CONSTRAINT navbar.top  == container.top
46 CONSTRAINT navbar.left == container.left
47 CONSTRAINT logo.centerY    == navbar.centerY
48 CONSTRAINT logo.left       == navbar.left + 24
49 CONSTRAINT navBtn2.centerY == navbar.centerY
50 CONSTRAINT navBtn2.right   == navbar.right - 20
51 CONSTRAINT navBtn1.centerY == navbar.centerY
52 CONSTRAINT navBtn1.right   == navBtn2.left - 10
53
54 // Hero sekcija
55 CONSTRAINT hero.top  == navbar.bottom
56 CONSTRAINT hero.left == container.left
57 CONSTRAINT heroTitle.centerX == hero.centerX
58 CONSTRAINT heroTitle.top     == hero.top + 60
59 CONSTRAINT heroSub.centerX   == hero.centerX
60 CONSTRAINT heroSub.top       == heroTitle.bottom + 16
61 CONSTRAINT heroCta.centerX   == hero.centerX
62 CONSTRAINT heroCta.top       == heroSub.bottom + 24
63
64 // Grid kartica
65 CONSTRAINT gridRow.top     == hero.bottom + 20
66 CONSTRAINT gridRow.centerX == container.centerX
67 CONSTRAINT featureA.top    == gridRow.top + 10
68 CONSTRAINT featureA.left   == gridRow.left + 10
69 CONSTRAINT featureB.top    == featureA.top
```

```
70 CONSTRAINT featureB.left   == featureA.right + 20
71 CONSTRAINT featureC.top    == featureA.top
72 CONSTRAINT featureC.left   == featureB.right + 20
73
74 // Ikone unutar kartica (treci nivo hijerarhije)
75 CONSTRAINT iconA.centerX == featureA.centerX
76 CONSTRAINT iconA.top     == featureA.top + 16
77 CONSTRAINT iconB.centerX == featureB.centerX
78 CONSTRAINT iconB.top     == featureB.top + 16
79 CONSTRAINT iconC.centerX == featureC.centerX
80 CONSTRAINT iconC.top     == featureC.top + 16
```

Primjer pokazuje tri nivoa gnježdenja: ikone su djeca feature kartica, koje su djeca gridRow panela. Solver propagira globalne koordinate kroz cijelo stablo – ako se pomjeri `hero` sekcija, `gridRow` i sve kartice i ikone unutar njih automatski slijede. Ovakav raspored u klasičnom CSS-u zahtijevao bi više deseti- na linija pozicioniranja, dok Layout Algebra opisuje iste odnose u 28 ograničenja čija semantika ostaje čitljiva i nakon izmjena.

Slika 3.4: Rezultat Primjera 4 – landing page s tri nivoa hijerarhije: navbar, hero sekcija i grid kartica s ugniježdenim ikonama

3.2 Analiza performansi

Performanse sistema su analizirane na više dimenzija: brzina rješavanja, reaktivnost korisničkog interfejsa i memorijska potrošnja.

Brzina rješavanja: Kiwi.js solver radi u $\mathcal{O}(m \cdot n)$ vremenskoj složenosti u prosjeku, gdje je m broj ograničenja i n broj varijabli. Za tipičan layout s do 20 elemenata (160 varijabli) i 50–100 ograničenja, rješavanje traje ispod 1 ms na modernom računaru. Za 60 fps animaciju dozvoljeno je do 16.7 ms po frame-u, što znači da solver ima veliku marginu čak i za složenije layoute.

Mjerenja su provedena u Chrome DevTools-u na Dell laptopu (16GB RAM):

Tablica 3.1: Izmjerena brzina rješavanja u zavisnosti od broja elemenata

Br. elemenata	Br. ograničenja	Br. varijabli	Vrijeme [ms]
5	15	40	< 0.1
10	30	80	< 0.3
20	60	160	< 0.8
50	150	400	< 3.0

Reaktivnost UI-a: Zahvaljujući inkrementalnoj prirodi Cassowary algoritma, promjene tokom prevlačenja se rješavaju gotovo trenutno. Jedino značajno kašnjenje može nastati pri re-parsiranju cijelog DSL koda, što se dešava samo kada korisnik promijeni DSL tekst. Ovo kašnjenje je u tipičnim slučajevima ispod 5 ms.

3.3 Poređenje s alternativnim pristupima

Tablica 3.2: Poređenje pristupa za definisanje layouta

Aspekt	**Apsolutno poz.**	**Flexbox/Grid**	**Layout Algebra**
Ekspresivnost	Eksplicitne koordinate	Apstraktni modeli	Logički odnosi
Fleksibilnost	Niska	Visoka	Srednja-visoka
Kriva učenja	Niska	Srednja	Srednja
Constraint konflikti	Ne postoje	Ne postoje	Mogući
Responsivnost	Teška	Ugrađena	Putem % dimenzija
Interaktivna man.	Direktna	Indirektna	Direktna + solver
Export mogućnosti	N/A	N/A	CSS/HTML

Layout Algebra pristup ima jedinstvenu prednost kada je u pitanju održavanje relacija. Kod apsolutnog pozicioniranja, ako se promijeni veličina jednog elementa, korisnik mora ručno ažurirati koordinate svih zavisnih elemenata. Kod Layout Algebra, promjena se automatski propagira kroz solver.

3.4 Ograničenja implementacije i diskusija

Svaki sistem ima ograničenja koja su važno transparentno dokumentovati.

Samo linearna ograničenja: Cassowary algoritam podržava isključivo linearna ograničenja. To znači da nije moguće izraziti, na primjer, ograničenje da širina elementa mora biti jednaka kvadratu neke druge dimenzije, ili da elementi moraju biti raspoređeni po kružnici. Za ovakve scenarije potrebni su nelinearni solveri koji su znatno sporiji i manje prikladni za real-time primjenu.

Kontradiktorni constrainti: Ako korisnik definira ograničenja koja se međusobno isključuju (npr. da element mora biti i na lijevom i na desnom rubu kontejnera istovremeno, uz REQUIRED prioritet), solver će prijaviti grešku. Sistem hvata ovu grešku i prikazuje opisnu poruku uz preporuku da se jedno ograničenje oslabi na WEAK. U budućim verzijama, sistem bi mogao automatski predložiti razrješenje konflikta.

Export aproksimacije: Flexbox export ne može uvijek savršeno replicirati raspored; Grid export je generalno tačniji. Apsolutni CSS export je uvijek tačan, ali ne generira responzivni CSS.

Responsivnost: Puna responsivna funkcionalnost (breakpoints, media queries) zahtijeva ručno dodefinisanje. Layout Algebra se fokusira na definiranje rasporeda za jednu veličinu prikaza.

Složenost hijerarhije: Duboke hijerarhije (više od 5 nivoa gnježdenja) mogu postati teško upravljive u DSL editoru. Rješenje bi bila grafička reprezentacija stabla hijerarhije s mogućnoštima prevlačenja čvorova, što je predviđeno za buduće verzije.

Zaključak

U ovom radu predstavljen je sistem **Layout Algebra** – interaktivni alat za upravljanje rasporedom elemenata web stranice zasnovan na programiranju ograničenjima. Sistem integrira namjenski domensko-specifični jezik za deklarativno opisivanje logičkih odnosa između elemenata, Cassowary solver za rješavanje sistema linearnih ograničenja te modernu React web aplikaciju za interaktivnu vizualizaciju i manipulaciju.

Ključni doprinosi ovog rada su:

1. **Dizajn i implementacija Layout Algebra DSL-a** – eksterni DSL s jasnom sintaksom za opisivanje elemenata, ograničenja, boja, hijerarhija i pravila izbjegavanja
2. **Integracija Cassowary solvera** – real-time rješavanje s prioritetima i interaktivno prevlačenje putem edit varijabli
3. **Dvosmjerna sinhronizacija** – korisnik može mijenjati DSL kod ili direktno manipulisati elementima
4. **Grupiranje i hijerarhije** – grupno prevlačenje s klampovanjem i hijerarhijska relacija roditelj-dijete
5. **Undo/Redo** – sveobuhvatan sistem historije s 50 koraka
6. **CSS export** – generisanje CSS/HTML u četiri formata

Sistem je implementiran kao production-ready prototip i demonstrira izvodljivost pristupa zasnovanog na ograničenjima za interaktivni web dizajn. Mjerenja performansi pokazuju da se za tipičan layout od 20 elemenata rješavanje odvija za manje od 1 ms, što je višestruko ispod praga koji je neophodan za fluidan 60 fps korisnički doživljaj.

Rezultati ovog rada otvaraju više pravaca za budući razvoj. U pogledu jezičnih proširenja, moguće je dodati podršku za nelinearne odnose putem iterativnog rješavanja (npr. Newton-Raphson) ili hibridnih solvera. U pogledu korisničkog iskustva, buduća verzija bi mogla uključiti syntax highlighting i autocomplete u DSL editoru, kao i grafički prikaz hijerarhije u obliku stabla. Integracija s popularnim dizajnerskim formatima (Figma, Sketch) otvorila bi mogućnosti za uvoz postojećih dizajna i dorade ih u Layout Algebra sistemu. Konačno, implementacija podrške za media queries i breakpointe učinila bi sistem upotrebljivim i za responsivni web dizajn.

Ovaj rad pokazuje da je pristup zasnovan na ograničenjima praktičan i upotrebljiv za web layout – ne samo kao istraživačka tema, već kao alat koji može olakšati svakodnevni rad front-end developera i web dizajnera.

A Sintaksa Layout Algebra DSL

Kompletna sintaksa Layout Algebra DSL-a (BNF notacija):

Listing A.1: BNF gramatika Layout Algebra DSL

```
program    ::= line*
line       ::= container | element | constraint | color
             | parent | avoid | comment | empty
comment    ::= "//" tekst
container  ::= "CONTAINER" broj broj
element    ::= "ELEMENT" id dim dim
dim        ::= broj | broj "%"
constraint ::= "CONSTRAINT" propref op expr_rhs [strength]
             | "CONSTRAINT" propref op broj      [strength]
propref    ::= id "." property
property   ::= "left" | "right" | "top" | "bottom"
             | "width" | "height" | "centerX" | "centerY"
op         ::= "==" | ">=" | "<="
expr_rhs   ::= propref [("*"|"/") broj] [("+"|"-") broj]
strength   ::= "WEAK" | "STRONG" | "REQUIRED"
color      ::= "COLOR" id "#" 6*hexdigit
parent     ::= "PARENT" id id
avoid      ::= "AVOID" id id
```

Primjer kompletnog DSL dokumenta:

Listing A.2: Kompletan DSL dokument primjer

```
// Primjer kompletnog layouta -- Layout Algebra
CONTAINER 800 600

ELEMENT header  800 60
ELEMENT sidebar 200 540
ELEMENT content 50% 540
ELEMENT footer  800 40
ELEMENT btn     120 40

COLOR header  #bd93f9
COLOR sidebar #6272a4
COLOR content #282a36
COLOR footer  #44475a
COLOR btn     #ff79c6

PARENT btn content

```

```
18 CONSTRAINT header.top      == container.top
19 CONSTRAINT header.left     == container.left
20 CONSTRAINT sidebar.top     == header.bottom
21 CONSTRAINT sidebar.left    == container.left
22 CONSTRAINT content.top     == header.bottom
23 CONSTRAINT content.left    == sidebar.right + 10
24 CONSTRAINT footer.bottom   == container.bottom
25 CONSTRAINT footer.left     == container.left
26 CONSTRAINT btn.centerX     == content.centerX  WEAK
27 CONSTRAINT btn.top         == content.top + 20
```

B Pregled strukture projekta

Sljedeća tabela prikazuje organizaciju fajlova u repozitoriju projekta i ulogu svakog modula:

Tablica B.1: Struktura projekta Layout Algebra

Putanja	Opis
`src/parser/parser.ts`	Implementacija DSL parsera; funkcija `parseDSL`
`src/solver/solver.ts`	Klasa `LayoutSolver`; integracija kiwi.js
`src/store/layoutStore.ts`	Zustand store; centralno upravljanje stanjem
`src/export/cssExport.ts`	Export modul (Absolute, Flexbox, Grid, HTML)
`src/components/Canvas.tsx`	Vizualni kanvas; drag-and-drop logika
`src/components/DSLEditor.tsx`	Textarea za DSL unos; prikaz grešaka
`src/components/ConstraintBuilder.tsx`	Grafički alat za kreiranje ograničenja
`src/components/ElementListPanel.tsx`	Lista elemenata; upravljanje hijerarhijom
`src/App.tsx`	Korijen React aplikacije; raspored panela
`vite.config.ts`	Vite konfiguracija
`package.json`	Zavisnosti projekta

Zavisnosti projekta koje su relevantne za ključne funkcionalnosti:

Tablica B.2: Ključne zavisnosti projekta

Paket	Verzija	Namjena
`react`	18.x	UI biblioteka
`react-dom`	18.x	DOM rendering
`kiwi.js`	1.x	Cassowary constraint solver
`zustand`	4.x	Upravljanje stanjem
`vite`	5.x	Build alat i dev server
`typescript`	5.x	Statička provjera tipova

Bibliografija

[1] K. Marriott, P. J. Stuckey, *Programming with Constraints: An Introduction*, MIT Press, Cambridge, MA, 1998.

[2] I. E. Sutherland, “Sketchpad: A Man-Machine Graphical Communication System”, *Proceedings of the Spring Joint Computer Conference*, pp. 329–346, 1963.

[3] G. J. Badros, A. Borning, P. J. Stuckey, “The Cassowary Linear Arithmetic Constraint Solving Algorithm”, *ACM Transactions on Computer-Human Interaction*, vol. 8, no. 4, pp. 267–306, 2001.

[4] Apple Inc., *Auto Layout Guide*, Apple Developer Documentation, 2013. Dostupno na: https://developer.apple.com/library/archive/documentation/UserExperience/Conceptual/AutolayoutPG/

[5] N. Ribaudo, *kiwi.js – A fast constraint solver for JavaScript/TypeScript*, 2023. Dostupno na: https://github.com/nicolo-ribaudo/kiwi.js

[6] M. Fowler, *Domain-Specific Languages*, Addison-Wesley Professional, 2010.

[7] Apple Inc., *Visual Format Language*, Apple Developer Documentation, 2014.

[8] Meta Platforms Inc., *React – A JavaScript library for building user interfaces*, 2024. Dostupno na: https://react.dev/

[9] D. Kato, *Zustand – Bear necessities for state management in React*, 2024. Dostupno na: https://github.com/pmndrs/zustand

[10] E. You, *Vite – Next Generation Frontend Tooling*, 2024. Dostupno na: https://vitejs.dev/

[11] A. Borning, “Thinglab – A Constraint-Oriented Simulation Laboratory”, PhD thesis, Stanford University, 1979.

[12] B. A. Myers, D. A. Giuse, R. B. Dannenberg et al., “Garnet: Comprehensive Support for Graphical, Highly Interactive User Interfaces”, *Computer*, vol. 23, no. 11, pp. 71–85, 1990.